CROSS-CHAIN DOCUMENT NOTARY SYSTEM

A Blockchain-Based Decentralized Notarization Platform

Course

CS 675 – Introduction to Blockchain and Cryptocurrency

Instructor

Prof. Subhasish Dhal

Team Members

Baheti Devkumar Gopal

Sahil

Vaishnavi Rai

Academic Year

2025 – 2026

Abstract

Traditional document notarization relies entirely on centralized, trusted third-parties — state-appointed notaries, law firms, or government bodies — making the process slow, expensive, geographically restricted, and vulnerable to single points of failure, fraud, or censorship. As global commerce and digital documentation accelerate, the need for a trustless, borderless, and immutable proof-of-existence system has never been more urgent.

This project proposes and fully implements a Cross-Chain Document Notary System leveraging Ethereum-based Smart Contracts and the Ethereum Virtual Machine (EVM). The system works by computing a cryptographic SHA-256 hash of a document directly in the user's browser — ensuring the raw file never leaves the client machine — and then permanently anchoring this fingerprint on an immutable distributed ledger. This creates a mathematically verifiable, timestamped, tamper-proof proof of document existence and ownership that requires no trusted intermediary to validate.

Furthermore, the architecture is designed from the ground up with cross-chain interoperability in mind. By deploying the same contract logic across multiple EVM-compatible chains (Ethereum Sepolia, Polygon Amoy, and local Hardhat nodes) and implementing an application-layer cross-chain registry, a document notarized on one blockchain can be independently detected and verified from a completely separate blockchain — eliminating the "Silo Problem" that plagues isolated blockchain ecosystems. The system demonstrates global uniqueness enforcement across network boundaries, a critical step toward Blockchain 3.0 interoperability.

Table of Contents

1. Introduction and Problem Statement 4
2. Objectives of the Project 5
3. Theoretical Foundation and Course Integration 6
4. System Architecture and Design 9
5. Smart Contract: DocumentNotary.sol 11
6. Frontend: index.html 15
7. Backend: Server and Deployment Scripts 17
8. Testing and Quality Assurance 19
9. Cross-Chain Architecture and Demo Workflow 21
10. Deployment Guide (Local and Testnet) 23
11. Real-World Use Cases and Applications 25
12. Security Analysis 26
13. Limitations and Future Work 27
14. Conclusion 28
15. References 29

1. Introduction and Problem Statement

1.1 Background

Document notarization is a legal process that authenticates the identity of a signing party and certifies the authenticity and integrity of a document. In the current system, this process requires a licensed notary public — a state-appointed official — who physically inspects documents, verifies identities, and stamps records. This traditional model, while legally recognized, carries several fundamental limitations:

- Centralization: The validity of a notarized document depends entirely on a single authority. If that authority is corrupt, hacked, or simply goes out of business, the integrity of all issued certificates is compromised.
- Geographic Restrictions: Notarization requires physical presence, making cross-border document authentication slow, costly, and logistically complex.
- Opacity and Forgery Risk: Traditional notary stamps can be physically forged. Without a public, immutable record, there is no easy way for a third party to independently verify the authenticity of a certificate.
- High Cost and Delay: Engaging a professional notary involves scheduling, travel, and fees, creating unnecessary friction for individuals and businesses.

1.2 The Blockchain Solution

Distributed Ledger Technology (DLT), and specifically Ethereum's Smart Contract platform, offers a compelling solution to all of the above problems. By replacing the human intermediary with a self-executing piece of code deployed on a globally distributed network, we achieve:

- Trustlessness: No single party controls the record. It is validated by thousands of independent nodes.
- Immutability: Once a transaction is written to the blockchain, it cannot be altered or deleted.
- Global Accessibility: Anyone with an internet connection and a cryptocurrency wallet can interact with the system, regardless of geography.
- Transparency: All notarization events are publicly auditable on-chain via blockchain explorers.

1.3 Project Scope

This project is scoped to build and demonstrate a fully functional Cross-Chain Document Notary System with the following capabilities:

1. A Solidity smart contract deployed on EVM-compatible chains that stores document hashes and their associated metadata.
2. A browser-based frontend (no framework, pure HTML/CSS/JS) that allows users to hash files locally, connect via MetaMask, and perform all notary actions.
3. A Node.js server to serve the frontend across a Local Area Network (LAN), enabling multi-user demonstrations.
4. A cross-chain detection mechanism that identifies document duplicates across separate blockchain networks at the application layer.
5. A comprehensive test suite with 15 test cases validating all contract functionality and edge cases.

2. Objectives of the Project

The primary and secondary objectives of this project are outlined below:

2.1 Primary Objectives

- Design and deploy a production-quality Solidity smart contract that permanently anchors cryptographic document fingerprints on-chain.
- Build a full-stack decentralized application (dApp) where users can notarize, verify, revoke, and transfer ownership of documents without any central server or authority.
- Implement cross-chain document collision detection, whereby a document already registered on one chain is flagged when a user attempts to re-register it on a different chain.
- Ensure user privacy: the actual document content never leaves the user's device; only its cryptographic hash is transmitted to the blockchain.

2.2 Secondary Objectives

- Demonstrate multi-user LAN-accessible blockchain interaction, enabling multiple peers to share the same Hardhat node and interact with the same contract concurrently.
- Provide a clear, well-documented codebase and deployment guide that can be extended for production use on public testnets (Sepolia, Polygon Amoy) or mainnet.
- Apply and demonstrate course concepts from CS 675 — including hashing, digital signatures, gas economics, and smart contract security — in a real-world context.
- Produce a comprehensive test suite to ensure all contract functions and edge cases are covered.

3. Theoretical Foundation and Course Integration

This project is deeply rooted in the concepts taught throughout CS 675. Each major feature of the system directly corresponds to a theoretical concept discussed in lectures and course materials.

3.1 Cryptographic Hashing and the Proof-of-Existence Model

The central mechanism of any document notary system on the blockchain is the cryptographic hash function. Instead of storing an entire document on-chain — which would be prohibitively expensive in terms of gas and would raise serious privacy concerns — our system stores only the document's hash, a fixed-length digital fingerprint.

As taught in CS 675, a secure cryptographic hash function (SHA-256 or Keccak-256) exhibits the following critical properties:

- Determinism: The same input always produces the same hash output.
- Collision Resistance: It is computationally infeasible to find two different inputs that produce the same hash.
- Avalanche Effect: A single-bit change in the input — even changing one comma in a 100-page document — produces a completely different hash output.
- One-Way (Pre-image Resistance): It is computationally infeasible to reconstruct the original document from its hash.

Our frontend uses the browser's built-in Web Crypto API (crypto.subtle.digest) to compute a SHA-256 hash of the uploaded file. This hash is then converted to bytes32 format (Solidity's native 32-byte type) and transmitted to the smart contract. The Avalanche Effect guarantees that any tampering with the original document — however minor — will produce a completely different hash that will not match the stored record, instantly exposing the forgery.

3.2 Smart Contracts and the Ethereum Virtual Machine (EVM)

Our notary contract is the practical embodiment of Nick Szabo's original concept of a "smart contract" — a self-executing agreement written in code that runs without any possibility of censorship, downtime, or third-party interference.

The DocumentNotary.sol contract is compiled from Solidity (version ^0.8.28) into EVM bytecode and deployed to the blockchain. Once deployed, its address becomes the global, immutable reference point for all notarization activity. Key EVM concepts applied:

- State Storage: The contract uses Solidity mapping data structures to permanently store document records in the EVM's persistent storage. This storage is replicated across every node on the network.
- Gas Economy: Every write operation (notarizing, revoking, transferring) consumes gas. Read operations (verifying, checking) are free (view functions). The Hardhat configuration enables an optimizer (runs: 200) to minimize gas consumption.
- The block.timestamp Variable: The contract records block.timestamp at the moment of notarization, providing a chain-enforced timestamp that is an integral part of the proof of existence.
- The chainid() Opcode: Used via inline assembly, this opcode reads the current network's Chain ID directly from the EVM context, enabling the contract to record which blockchain it is operating on.

3.3 Key Management, Digital Wallets, and Ownership

The concept of ownership in our system is grounded in Ethereum's public-key cryptography model. Every notarization is tied to the msg.sender — the Ethereum address of the caller, which is derived from their private key.

- Externally Owned Accounts (EOAs): Users interact via a Hot Wallet such as MetaMask, which manages their private key and signs every transaction. The signature proves that only the holder of that private key authorized the action.
- The onlyOwner Modifier: Sensitive operations like revokeDocument and transferOwnership are gated by a Solidity modifier that checks require(documents[_docHash].owner == msg.sender). This is a cryptographically enforced access control mechanism — not just a software check.
- Non-repudiation: Because every transaction is signed with the user's private key and permanently recorded on-chain, the owner cannot later deny having notarized a document. This is equivalent to a handwritten signature but mathematically verifiable.

3.4 Decentralization and the P2P Network

The Hardhat local development node simulates Ethereum's peer-to-peer network on a single machine or LAN. When bound to 0.0.0.0 (all network interfaces), it behaves like a true distributed node accessible by any device on the same WiFi network. In the full production scenario, the contract is deployed to Sepolia or Polygon Amoy — public testnets where nodes are operated by independent parties worldwide.

3.5 Cross-Chain Interoperability (Blockchain 3.0)

As discussed in the "Recent Trends in Blockchain" lectures, Blockchain 3.0 is defined by interoperability — the ability for separate, isolated blockchain networks to communicate and share state. Our project implements a practical demonstration of application-layer cross-chain messaging.

The system maintains a global registry of known contract deployments across multiple chains. When a user connects to Chain B (e.g., Polygon), the application simultaneously queries all contracts on Chain A (e.g., Ethereum) for the same document hash. If the hash is found on another chain, a CROSS-CHAIN COLLISION DETECTED error is raised, preventing the same document from being notarized on multiple chains without the owner's explicit re-notarization.

4. System Architecture and Design

The system is structured as a three-layer decentralized application (dApp). Each layer has a clearly defined responsibility, and they communicate through well-defined interfaces.

4.1 Architectural Overview

Layer	Component(s)	Responsibility
Layer 1: On-Chain	DocumentNotary.sol	Permanent, immutable storage of document hashes, timestamps, and ownership records.
Layer 2: Web3 Bridge	ethers.js v6, deploy.js, hardhat.config.js	Connects the frontend to the blockchain; handles contract ABI, transaction signing, and event listening.
Layer 3: Frontend	index.html, server.js	User-facing interface for file hashing, MetaMask connection, and displaying notarization results.

4.2 Data Flow

The complete flow of a document notarization request proceeds as follows:

6. User selects a file in the browser (index.html).
7. The frontend reads the file as an ArrayBuffer and computes its SHA-256 hash using the Web Crypto API. The raw file is never sent anywhere.
8. The frontend checks the cross-chain registry — it queries all known contract deployments on all registered chains to see if this hash has been notarized on any chain.
9. If no collision is found, the user fills in optional metadata (document title, description) and clicks "Notarize."
10. MetaMask intercepts the transaction request, shows the user the gas cost estimate, and asks for their digital signature.
11. Upon approval, ethers.js submits the signed transaction to the Hardhat node (or public testnet RPC).

12. The EVM executes the notarizeDocument function, writes the Document struct to persistent storage, increments totalDocuments, and emits a DocumentNotarized event.
13. The transaction receipt is confirmed. The frontend reads the event data and displays the Block number, Transaction Hash, Timestamp, Gas Used, and Chain ID to the user.

4.3 Project File Structure

File / Directory	Description
contracts/DocumentNotary.sol	Core Solidity smart contract — the heart of the system.
index.html	Single-page frontend application (~1,461 lines). All HTML, CSS, and JavaScript in one file.
server.js	Lightweight Node.js HTTP server that serves index.html to all devices on the LAN.
scripts/deploy.js	Hardhat deployment script. Deploys contract, runs a smoke test, and auto-updates index.html with the new contract address.
scripts/checkDeploy.js	Utility script to verify an existing deployment is still live and functional.
scripts/findMyContract.js	Helper to search for a contract address in the local Hardhat node's history.
test/DocumentNotary.js	Mocha/Chai test suite containing 15 comprehensive test cases.
hardhat.config.js	Hardhat configuration: Solidity version, optimizer settings, and network definitions.
.env / .env.example	Environment variable templates for RPC URLs and private keys (never committed to Git).
artifacts/	Auto-generated compilation artifacts (ABI, bytecode). Used by the frontend to instantiate the contract.

5. Smart Contract: DocumentNotary.sol

The smart contract is the authoritative, immutable core of the system. It is written in Solidity ^0.8.28 and compiled with the optimizer enabled (200 runs) to minimize gas consumption. Every design decision was made with security, minimalism, and gas efficiency in mind.

5.1 The Document Struct

All data about a notarized document is stored in a single Solidity struct:

```
struct Document {
    bytes32  docHash;     // SHA-256 hash of the document
    address  owner;       // Current owner / notarizer (msg.sender)
    uint256  timestamp;   // block.timestamp at time of notarization
    string   metadata;    // Optional description / filename
    bool     isRevoked;   // Has the owner invalidated this record?
    uint256  chainId;     // Chain ID where it was notarized
}
```

Each field serves a specific purpose. The docHash is the primary key — a unique identifier derived from the document's content. The owner links the document to a cryptographic identity. The timestamp provides legally meaningful proof of when the document existed. The metadata is an optional human-readable label. The isRevoked flag implements a soft-delete mechanism. The chainId records which blockchain network the notarization occurred on, enabling cross-chain verification.

5.2 State Variables

```
// docHash => Document struct
mapping(bytes32 => Document) private documents;

// owner address => list of their document hashes
mapping(address => bytes32[]) private ownerDocuments;

// Total count of unique documents notarized
uint256 public totalDocuments;
```

Both mappings are declared private to prevent direct external access. All data must be retrieved through the defined getter functions, which can perform additional validation. totalDocuments is public because it is a useful, non-sensitive metric.

5.3 Events

Events are the primary mechanism by which on-chain activity is communicated to the off-chain frontend. All three events are indexed to allow efficient querying:

```
event DocumentNotarized(
    bytes32 indexed docHash,
    address indexed owner,
    uint256 timestamp,
    string  metadata,
    uint256 chainId
);

event DocumentRevoked(
    bytes32 indexed docHash,
    address indexed owner,
    uint256 timestamp
);

event OwnershipTransferred(
    bytes32 indexed docHash,
    address indexed previousOwner,
    address indexed newOwner,
    uint256 timestamp
);
```

5.4 Modifiers

Three Solidity modifiers enforce preconditions on function calls, making the code DRY (Don't Repeat Yourself) and the access logic explicit and auditable:

```
modifier onlyOwner(bytes32 _docHash) {
    require(documents[_docHash].owner == msg.sender,
            "Not the document owner");
    _;
}

modifier documentExists(bytes32 _docHash) {
```

```
    require(documents[_docHash].timestamp != 0,
            "Document not found");
    _;
}

modifier notRevoked(bytes32 _docHash) {
    require(!documents[_docHash].isRevoked,
            "Document has been revoked");
    _;
}
```

5.5 Core Functions

The contract exposes six public functions. The table below documents each one:

Function	Type	Modifiers	Description
notarizeDocument(bytes32, string)	Write	—	Registers a new document hash. Rejects duplicates unless previously revoked. Increments totalDocuments for brand-new entries. Uses inline assembly to capture the current chainId.
verifyDocument(bytes32)	View	documentExists	Returns the full Document struct: owner address, timestamp, metadata string, revocation status, and chain ID.
isNotarized(bytes32)	View	—	A lightweight boolean check: returns true if a document hash has ever been notarized (even if later revoked).
revokeDocument(bytes32)	Write	onlyOwner, documentExists, notRevoked	Marks a document as revoked (isRevoked = true). This action is permanent and irreversible. Emits DocumentRevoked.
transferOwnership(bytes32, address)	Write	onlyOwner, documentExists, notRevoked	Transfers the owner field to a new address. Adds the hash to the new owner's ownerDocuments list. Emits OwnershipTransferred.
getOwnerDocuments(address)	View	—	Returns the full bytes32[] array of all document hashes associated with a given owner address.
getOwnerDocumentCount(address)	View	—	Returns the uint256 count of documents associated with a given owner address. Gas-

Function	Type	Modifiers	Description
			efficient alternative to fetching the full array.

5.6 Key Implementation Details

5.6.1 Re-Notarization Logic

A design decision was made to allow re-notarization of previously revoked documents. This mirrors real-world notary practice where a document can be re-certified after an old certificate is invalidated. The require statement reflects this:

```
require(
    documents[_docHash].timestamp == 0 || documents[_docHash].isRevoked,
    "Document already notarized"
);
```

The totalDocuments counter only increments for brand-new documents (timestamp == 0) to avoid inflating the count with re-notarizations.

5.6.2 Inline Assembly for Chain ID

The EVM's chainid() opcode is accessed via Solidity inline assembly, which is the most gas-efficient method:

```
uint256 currentChainId;
assembly { currentChainId := chainid() }
```

6. Frontend: index.html

The frontend is a single, self-contained HTML file of approximately 1,461 lines. It requires no build step, no npm install, and no JavaScript framework. This design choice makes deployment trivially simple — it can be served by any static file host or opened directly in a browser with a local RPC endpoint.

6.1 Technology Stack

Technology	Usage
HTML5 / CSS3	Page structure and all visual styling. No external CSS framework.
Vanilla JavaScript (ES2022)	All application logic, event handling, and DOM manipulation.
ethers.js v6 (CDN)	Web3 library for MetaMask connection, contract instantiation, transaction submission, and event parsing.
Web Crypto API	Browser-native SHA-256 hashing of uploaded files. No external library required.
Google Fonts (CDN)	"Syne" (headings) and "Space Mono" (body) for the cyberpunk-inspired aesthetic.

6.2 Visual Design

The UI adopts a dark, cyberpunk-inspired design language with a CSS custom property (variable) theme system. This makes the entire colour scheme trivially customizable. Key design features:

- Dark base background (#0a0a0f) with subtle CSS grid overlay for a technical aesthetic.
- A fixed radial gradient "glow orb" in the top-right corner using the accent purple (#7b2fff) for visual depth.
- Colour-coded status indicators: cyan (#00e5ff) for primary actions, green (#00c87a) for success, red (#ff4757) for errors, and orange (#ffa726) for warnings.
- Card-based layout with glass-morphism-style borders and subtle surface colours.

6.3 Key Frontend Features

6.3.1 MetaMask Connection and Network Management

On clicking Connect, the frontend uses ethers.js to request access to the user's MetaMask wallet. Crucially, it also auto-configures MetaMask with the correct custom network parameters for the Hardhat local node:

- Network Name: DocNotary LAN (auto-detected IP)
- RPC URL: http://[HOST_IP]:8545
- Chain ID: 31337 (Hardhat's default)

This eliminates the need for users to manually configure MetaMask, reducing friction in a multi-user demo scenario.

6.3.2 Client-Side SHA-256 Hashing

When a user drags and drops or selects a file, the following process occurs entirely in the browser:

14. The File object is read as an ArrayBuffer using the FileReader API.
15. crypto.subtle.digest('SHA-256', buffer) computes the hash asynchronously.
16. The resulting ArrayBuffer is converted to a hexadecimal string prefixed with 0x.
17. The hash is displayed in the UI and stored for submission to the contract.

6.3.3 Real-Time Duplicate Detection

Before a user can submit a notarization transaction, the frontend calls isNotarized(bytes32) — a free view function — to check if the hash already exists on the connected chain. If it does, the full document record (owner, timestamp, metadata) is fetched via verifyDocument(bytes32) and displayed to the user with a warning banner, preventing a wasted gas transaction.

6.3.4 Cross-Chain Registry Query

The frontend maintains a registry of all known contract deployments across different chains. When connected to any chain, it instantiates read-only ethers.js providers for every other registered chain and concurrently queries each one for the document hash using Promise.all. If any chain returns a positive isNotarized result, the UI displays a CROSS-CHAIN COLLISION DETECTED alert.

6.3.5 Contract ABI Reference Panel

A dedicated section in the frontend renders the full Application Binary Interface (ABI) of the deployed contract, formatted as a readable reference. This allows developers to directly understand how to interact with the contract programmatically without consulting external documentation.

7. Backend: Server and Deployment Scripts

7.1 LAN HTTP Server (server.js)

The server.js file is a minimal Node.js HTTP server built using only the built-in http, fs, path, and os modules — no npm dependencies required. Its sole purpose is to serve index.html and any associated static assets to all devices on the LAN.

Key implementation details:

- Listens on 0.0.0.0:3000 (all network interfaces), making it accessible from any device on the same WiFi/LAN.
- Supports correct MIME types for HTML, CSS, JS, JSON, PNG, JPG, SVG, and ICO files.
- Sets Access-Control-Allow-Origin: * on all responses to prevent CORS issues when ethers.js loads from a CDN.
- On startup, enumerates all IPv4 network interfaces via os.networkInterfaces() and prints shareable LAN URLs to the console.

7.2 Deployment Script (scripts/deploy.js)

The deploy.js script is executed via Hardhat's runtime environment (HRE) and performs the following steps automatically:

18. Logs the deployer's Ethereum address and ETH balance.
19. Calls hre.ethers.getContractFactory("DocumentNotary") and deploys the contract using the first available signer.
20. Awaits deployment confirmation and logs the deployed contract address.
21. Executes a smoke test: notarizes a sample document hash directly from the deployer account and immediately verifies it, confirming the contract is functional.
22. Prints all LAN IP addresses with shareable URLs in the format http://[IP]:3000?contract=[ADDRESS].
23. Auto-updates index.html: reads the file, uses a regex to replace the contract address constant for the current network (localhost or sepolia), and writes the file back. This eliminates manual copy-pasting of the contract address.

7.3 Hardhat Configuration (hardhat.config.js)

The hardhat.config.js file configures the Hardhat development environment with the following settings:

Configuration	Value / Explanation
Solidity Version	0.8.28 — latest stable, includes all modern language features.
Optimizer	Enabled with 200 runs. Balances deployment cost vs. call cost for average usage.
Hardhat Network Chain ID	31337 (default) or overridden via CHAIN_ID environment variable for cross-chain simulation.
Localhost Network	URL configurable via LOCAL_RPC env var (default: http://127.0.0.1:8545).
Sepolia Network	Configured via SEPOLIA_RPC_URL and PRIVATE_KEY environment variables.
Amoy Network	Polygon's testnet, configured via AMOY_RPC_URL and PRIVATE_KEY.
Gas Reporter	Enabled when REPORT_GAS=true env var is set. Reports USD cost of each function.

8. Testing and Quality Assurance

The test suite (test/DocumentNotary.js) is written using the Mocha testing framework and Chai assertion library, both integrated into Hardhat via the @nomicfoundation/hardhat-toolbox package. The suite contains 15 individual test cases organized into 5 describe blocks.

8.1 Test Infrastructure

- Before each test, a fresh instance of the contract is deployed using ethers.getContractFactory and deploy(), ensuring complete test isolation.
- Three signers are obtained from ethers.getSigners(): owner, user1, and user2, simulating interactions from different Ethereum addresses.
- A helper function fakeHash(str) uses ethers.keccak256(ethers.toUtf8Bytes(str)) to generate deterministic bytes32 test hashes.

8.2 Test Case Summary

#	Test Group / Name	What It Validates
1	Deployment	Contract deploys with totalDocuments = 0.
2	notarizeDocument	A new document is notarized and emits DocumentNotarized event with correct args.
3	notarizeDocument	Attempting to notarize an already-notarized document reverts with 'Document already notarized'.
4	notarizeDocument	Passing bytes32(0) as the hash reverts with 'Invalid document hash'.
5	notarizeDocument	Owner's document list is correctly populated after multiple notarizations.
6	verifyDocument	Correct owner, metadata, and isRevoked = false are returned after notarization.
7	verifyDocument	Querying a non-existent hash reverts with 'Document not found'.
8	isNotarized	Returns false before notarization and true after.
9	revokeDocument	Owner can revoke their document; event is emitted; isRevoked becomes true.
10	revokeDocument	A non-owner cannot revoke; reverts with 'Not the document owner'.

#	Test Group / Name	What It Validates
11	revokeDocument	Revoking an already-revoked document reverts with 'Document has been revoked'.
12	transferOwnership	Ownership transfers correctly; OwnershipTransferred event emitted with correct args.
13	transferOwnership	Transfer to the zero address reverts with 'Invalid new owner'.
14	transferOwnership	Self-transfer reverts with 'Already the owner'.
15	transferOwnership	After transfer, the new owner's document list contains the transferred hash.

8.3 Running the Tests

Tests are executed via:

```
npx hardhat test

# With gas reporting enabled:
REPORT_GAS=true npx hardhat test
```

All 15 tests pass successfully, confirming the contract behaves correctly for all defined scenarios including happy paths, access control enforcement, and edge case input validation.

9. Cross-Chain Architecture and Demo Workflow

The cross-chain capability is the most distinctive feature of this project and the concept that aligns it with the Blockchain 3.0 vision discussed in the course's "Recent Trends" module.

9.1 The Silo Problem

Without cross-chain communication, each blockchain is an isolated island. A document notarized on Ethereum has no way of being automatically "known" to a Polygon smart contract. This means a malicious actor could re-notarize the same document on a different chain, falsely claiming priority of ownership.

9.2 Our Application-Layer Cross-Chain Solution

Our system solves this at the application layer (in the frontend) rather than requiring a dedicated cross-chain bridge protocol (like LayerZero or Chainlink CCIP). The approach is as follows:

24. The frontend maintains a cross-chain registry — a JavaScript object that maps Chain IDs to their corresponding deployed contract addresses and RPC endpoints.
25. When a user connects to any chain, the frontend instantiates read-only ethers.js JsonRpcProvider instances for every other chain in the registry.
26. For each proposed document hash, it concurrently calls isNotarized(hash) on every other chain's contract using Promise.all.
27. If any call returns true, the UI displays a prominent CROSS-CHAIN COLLISION DETECTED warning and blocks the notarization.

9.3 Live Demo Procedure

The following procedure is used for the live demonstration in front of the instructor:

Setup Phase

- Terminal 1: Start the main blockchain (Chain 1, Port 8545): npx hardhat node
- Terminal 2: Start a second blockchain (Chain 2, Port 8546): $env:CHAIN_ID="31338"; npx hardhat node --port 8546

- Terminal 3: Deploy to Chain 1: npx hardhat run scripts/deploy.js --network localhost
- Terminal 3: Deploy to Chain 2: $env:LOCAL_RPC="http://127.0.0.1:8546"; npx hardhat run scripts/deploy.js --network localhost
- Terminal 3: Start the web server: npx http-server -p 8080 -a 0.0.0.0

Demo Sequence

28. Step 1 — Normal Notarization: Connect MetaMask to Chain 1 (localhost:8545). Upload a PDF and notarize it. Observe the transaction confirmation, block number, gas used, and timestamp displayed in the UI.
29. Step 2 — Same-Chain Duplicate Prevention: Attempt to upload the same PDF again on Chain 1. Observe the system immediately blocking the upload with a warning showing the original owner's address and notarization timestamp.
30. Step 3 — Cross-Chain Collision Detection: Switch MetaMask to Chain 2 (localhost:8546, Chain ID 31338). Attempt to upload the same PDF. Observe the CROSS-CHAIN COLLISION DETECTED error — the document was found on Chain 1, and the system blocks re-notarization on Chain 2.

This three-step demonstration proves: (a) the basic notarization works, (b) same-chain duplicates are blocked at the contract level, and (c) cross-chain duplicates are detected at the application layer.

10. Deployment Guide (Local and Testnet)

10.1 Prerequisites

Requirement	Details
Node.js (LTS)	Required for Hardhat and the local server. Verify with: node -v
npm	Node Package Manager, bundled with Node.js. Verify with: npm -v
MetaMask	Browser extension for all Chromium-based browsers and Firefox.
Git	For cloning the repository.
Alchemy / Infura Account (Testnet only)	Free-tier account provides RPC URLs for Sepolia and Polygon networks.

10.2 Local Development Setup

31. Clone or extract the project repository.
32. Install dependencies: npm install
33. Compile the smart contract: npx hardhat compile
34. Run the test suite: npx hardhat test
35. Start the local Hardhat blockchain node (Terminal 1): npm run node
36. In a second terminal, deploy the contract: npm run deploy:local
37. Note the contract address printed in the output.
38. Start the LAN HTTP server (Terminal 3): npm run serve
39. Open a browser and navigate to http://localhost:3000
40. Connect MetaMask. Import one of the Hardhat test accounts using the private key shown in the node terminal (each account has 10,000 test ETH).

10.3 Multi-User LAN Setup

To allow multiple people to participate in the same demo:

- Run the Hardhat node and frontend server on one laptop as described above.
- Share the URL shown in the npm run serve output: http://[YOUR_IP]:3000?contract=[CONTRACT_ADDRESS]

- Each participant must have MetaMask installed. The frontend auto-configures the custom network on connect.
- Distribute Hardhat test account private keys to participants so they have test ETH for transactions.

10.4 Testnet Deployment (Sepolia or Polygon Amoy)

41. Copy .env.example to .env.
42. Populate SEPOLIA_RPC_URL (or AMOY_RPC_URL) with your Alchemy RPC endpoint.
43. Populate PRIVATE_KEY with your MetaMask account's private key. Never commit this file.
44. Get free testnet ETH from a faucet (Alchemy Sepolia Faucet or Polygon Amoy Faucet).
45. Deploy: npm run deploy:sepolia (or: npx hardhat run scripts/deploy.js --network amoy)
46. Host index.html on GitHub Pages, Vercel, or Netlify (free static hosting).
47. Share the hosted URL with the contract address: https://your-site.vercel.app/?contract=0x...

11. Real-World Use Cases and Applications

The Cross-Chain Document Notary System has broad applicability across multiple industries and use cases:

11.1 Intellectual Property Protection

Artists, writers, developers, and inventors can timestamp their creative works on the blockchain before filing for formal patents or copyrights. The immutable timestamp provides evidence of prior art in disputes, without requiring expensive legal fees upfront. The file content never leaves the user's device, protecting confidentiality during the patent application process.

11.2 Academic Credential Verification

Universities can notarize degree certificates, transcripts, and diplomas on a public blockchain. Employers can independently verify the authenticity of a credential in seconds by hashing the provided PDF and querying the contract — no phone calls to registrars, no verification fees, and no risk of forged certificates. Cross-chain deployment means any employer on any supported network can verify credentials from a globally deployed registry.

11.3 Legal Document Certification

Wills, contracts, property deeds, and affidavits can be notarized on-chain. The revocation mechanism (revokeDocument) allows old versions of documents to be formally invalidated. The ownership transfer (transferOwnership) is directly applicable to property deed transfers, where the legal ownership of a document corresponds to the blockchain ownership.

11.4 Supply Chain and Enterprise Logistics

Enterprises using private chains (like Hyperledger Fabric) can anchor supply chain manifests, certificates of authenticity, and quality assurance reports to a public chain via this system. Consumers can scan a QR code, retrieve the document hash, and verify it against the public blockchain — bridging the gap between private enterprise chains and public consumer verification.

11.5 Healthcare Records

Patient medical records, test results, and prescriptions can be notarized at the time of creation. The hash anchors the document to a specific point in time. The privacy-preserving design (only the hash is on-chain) ensures HIPAA compliance — sensitive patient data never touches the blockchain.

11.6 Insurance Claims

Insurance companies and claimants can notarize evidence (photos, repair estimates, medical bills) at the time of the incident. This prevents retroactive manipulation of evidence and creates an auditable, timestamped record that simplifies claim adjudication.

12. Security Analysis

Security is a foundational concern in smart contract development. The following analysis covers the security properties of our system and the vulnerabilities that were considered and mitigated.

12.1 Reentrancy

Reentrancy attacks (famously exploited in the 2016 DAO hack) occur when an external contract call allows an attacker to re-enter the calling function before it completes. Our contract is not vulnerable to reentrancy because: (a) it makes no external calls whatsoever, and (b) all state is updated before any event is emitted. There are no .call() or .transfer() invocations.

12.2 Access Control

All sensitive state mutations (revokeDocument, transferOwnership) are gated by the onlyOwner modifier, which uses require(documents[_docHash].owner == msg.sender). This check is enforced at the EVM level — it cannot be bypassed by the frontend, a relay, or any other off-chain actor.

12.3 Integer Overflow / Underflow

Solidity ^0.8.x introduces built-in overflow/underflow protection. Arithmetic operations revert automatically if they would exceed their type's range. The totalDocuments counter can hold values up to 2^256 - 1, making overflow impossible in practice.

12.4 Hash Collision Resistance

The system's security model relies on SHA-256 collision resistance. As of 2026, SHA-256 has no known practical collisions and is approved by NIST for cryptographic use. The 256-bit output space (2^256 possible values) makes brute-force collision finding computationally infeasible.

12.5 Private Key Security

The .env file containing the deployer's private key is listed in .gitignore and must never be committed to source control. The .env.example file provides a template without real credentials. During local testing, only Hardhat's automatically generated test accounts are used — these are deterministic, publicly known keys that hold no real value.

12.6 Front-Running

Since document hashes are deterministic, a miner who sees a pending notarize transaction in the mempool could theoretically copy the hash and submit a transaction with higher gas to be mined first, stealing the "first notarizer" status. Mitigation strategies include: commitment schemes (hash-reveal patterns), or using a private mempool. This is acknowledged as a limitation and a direction for future work.

13. Limitations and Future Work

13.1 Current Limitations

- Application-Layer Cross-Chain: The current cross-chain detection is implemented in the frontend JavaScript, not in a dedicated on-chain bridge or relay. It requires the user's browser to be able to connect to multiple RPC endpoints simultaneously. A production-grade solution would use a dedicated messaging protocol such as Chainlink CCIP, LayerZero, or Axelar.
- Front-Running Vulnerability: As discussed in Section 12.6, a miner or validator could potentially front-run a notarization transaction. A commit-reveal scheme would mitigate this.
- Metadata Stored On-Chain: The metadata string is stored in contract storage, which is the most expensive form of EVM storage. For long metadata strings, this becomes costly. A better approach would be to store a content-addressed hash (CIDv1 from IPFS) on-chain and store the full metadata off-chain on IPFS.
- No Document Versioning: The current system treats each unique hash as a separate document. There is no native concept of "version 2 of document X." A versioning system linking document hashes in a chain would be a valuable addition.
- Centralized Frontend: While the contract is fully decentralized, the frontend is served from a centralized HTTP server. Hosting the frontend on IPFS would make the entire system decentralized end-to-end.

13.2 Future Work

- On-Chain Cross-Chain Bridge Integration: Integrate Chainlink CCIP or LayerZero to enable trustless, on-chain verification of cross-chain document records without relying on the application layer.
- IPFS Integration for Metadata: Store document metadata as JSON on IPFS and record only the CID on-chain, reducing gas costs for metadata-heavy use cases.
- ERC-721 NFT Notarization: Represent each notarized document as a non-fungible token (NFT). This would leverage the existing ERC-721 standard for ownership and transfer, enabling notarized documents to be traded on existing NFT marketplaces.
- Zero-Knowledge Proof Integration: Use ZK-SNARKs to allow a prover to demonstrate knowledge of a document's content without revealing the document itself — a privacy-preserving extension of the current hash-based model.

- DAO-Based Governance: Implement a Decentralized Autonomous Organization (DAO) that governs which chains are added to the cross-chain registry, with token-weighted voting for governance decisions.
- Mobile Application: A React Native mobile app that allows users to notarize documents directly from their phone's camera or file system.

14. Conclusion

This project set out to build a trustless, borderless, and immutable document notarization system using blockchain technology — and it has successfully delivered on all of those objectives.

The DocumentNotary.sol smart contract provides a rock-solid, gas-efficient on-chain foundation. Its six public functions cover the complete lifecycle of a notarized document: creation, verification, revocation, and ownership transfer. The 15-test suite validates every function and edge case, giving high confidence in the correctness of the implementation.

The index.html frontend provides an accessible, intuitive interface that abstracts away the complexity of blockchain interaction. Users do not need to understand hashing algorithms, gas economics, or EVM internals to successfully notarize a document. The integration with MetaMask, automatic network configuration, real-time duplicate detection, and cross-chain collision detection all contribute to a polished, production-quality user experience.

The cross-chain demonstration is particularly significant. By running two independent Hardhat nodes simultaneously and showing that a document notarized on one chain is detected and blocked on the other, the project provides a concrete, working illustration of the interoperability challenges and solutions discussed in the Blockchain 3.0 module of CS 675.

Looking forward, the system is well-positioned for enhancement. Integration with dedicated cross-chain protocols (CCIP, LayerZero), IPFS-based metadata storage, ERC-721 tokenization, and ZK-proof-based privacy extensions would transform this academic prototype into a production-grade, globally deployable platform.

By combining the immutable security of the Ethereum Virtual Machine, the cryptographic guarantees of SHA-256 hashing, the access control of public-key cryptography, and the borderless potential of cross-chain communication, the Cross-Chain Document Notary System embodies the complete arc of blockchain evolution — from Bitcoin's simple ledger, through Ethereum's programmable contracts, to the interoperable, globally networked decentralized future.

15. References

48. Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. bitcoin.org.
49. Buterin, V. (2013). Ethereum White Paper: A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum Foundation.
50. Szabo, N. (1994). Smart Contracts. Nick Szabo's Papers and Concise Tutorials.
51. Wood, G. (2014). Ethereum: A Secure Decentralised Generalised Transaction Ledger. Ethereum Foundation. (The "Yellow Paper")
52. National Institute of Standards and Technology (NIST). (2015). FIPS PUB 180-4: Secure Hash Standard (SHS). U.S. Department of Commerce.
53. Hardhat Documentation. (2024). Hardhat: Ethereum Development Environment. hardhat.org.
54. Ethers.js Documentation. (2024). ethers v6. docs.ethers.org.
55. OpenZeppelin. (2024). Contracts: Secure Smart Contract Library. docs.openzeppelin.com.
56. Solidity Documentation. (2024). Solidity ^0.8.28 Language Reference. docs.soliditylang.org.
57. Chainlink. (2024). Cross-Chain Interoperability Protocol (CCIP). docs.chain.link.
58. LayerZero. (2024). Omnichain Interoperability Protocol. layerzero.network.
59. Web3 Foundation. (2023). Polkadot: Vision for a Heterogeneous Multi-Chain Framework.
60. IPFS Documentation. (2024). InterPlanetary File System. docs.ipfs.tech.
61. Atzei, N., Bartoletti, M., & Cimoli, T. (2017). A Survey of Attacks on Ethereum Smart Contracts. Proceedings of POST 2017.
62. Dhal, S. (2025). CS 675: Introduction to Blockchain and Cryptocurrency — Lecture Slides and Course Notes.

— End of Report —

Baheti Devkumar Gopal | Sahil | Vaishnavi Rai
CS 675 — Cross-Chain Document Notary System — 2025-26